

Home (https://learnopencv.com) > Deep Learning (https://learnopencv.com/category/deep-learning/) > Transfer Learning For PyTorch Image Classification

Sunita Nayak (https://learnopencv.com/author/snayak/) on May 20, 2019

Transfer Learning For PyTorch Image Classification

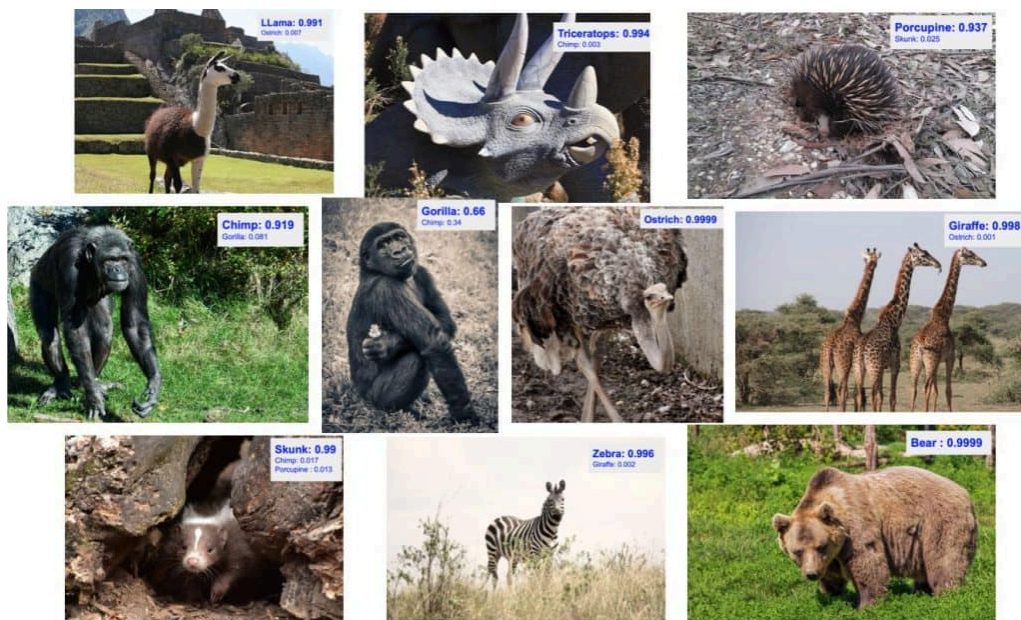
We describe how to do image classification in PyTorch. We use a subset of CalTech256 dataset to classify 10 different kinds of animals.

Deep Learning (https://learnopencv.com/category/deep-learning/)

Getting Started with PyTorch (https://learnopencv.com/category/getting-started-with-pytorch/)

Image Classification (https://learnopencv.com/category/image-classification/)

PyTorch (https://learnopencv.com/category/pytorch/)



Transfer Learning has revolutionized the way we approach image classification in PyTorch. Recently PyTorch has gained a lot of popularity because of its ease of usage and learning. Andrej Karpathy, Senior Director of AI at Tesla, said the following in his tweet.



Andrej Karpathy ✓
@karpathy

Following

I've been using PyTorch a few months now
and I've never felt better. I have more energy.
My skin is clearer. My eye sight has
improved.

Jokes apart, PyTorch is very transparent and can help researchers and data scientists achieve high productivity and reliable results.

This article is part of the following series:

PyTorch for Beginners

PyTorch for Beginners: Basics (/pytorch-for-beginners-basics/)

PyTorch for Beginners: Image Classification using Pre-trained models (/pytorch-for-beginners-image-classification-using-pre-trained-models/)

Image Classification using Transfer Learning in PyTorch (/image-classification-using-transfer-learning-in-pytorch/)

PyTorch Model Inference using ONNX and Caffe2 (/pytorch-model-inference-using-onnx-and-caffe2/)

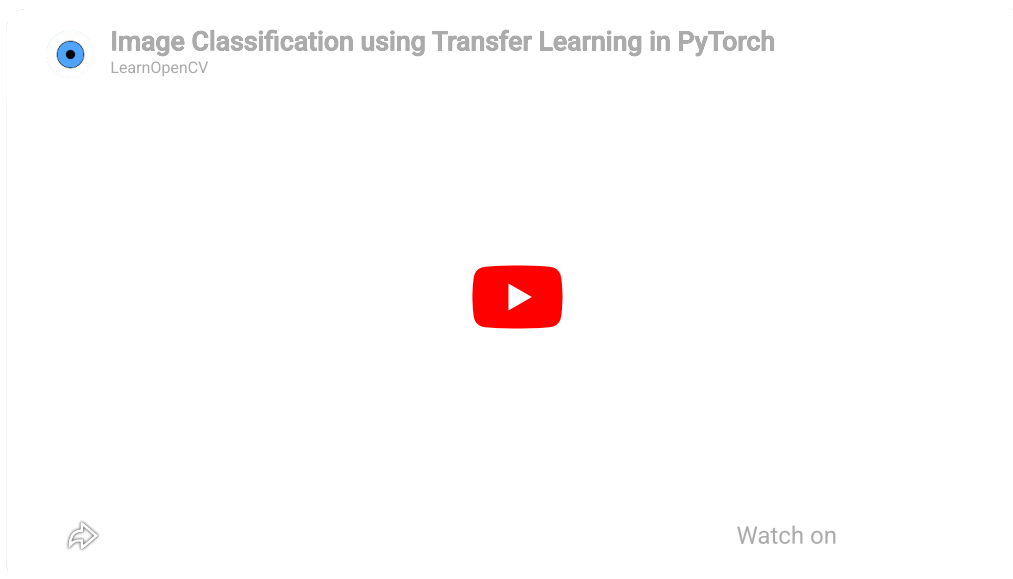
PyTorch for Beginners: Semantic Segmentation using torchvision (/pytorch-for-beginners-semantic-segmentation-using-torchvision/)

Object Detection (/faster-r-cnn-object-detection-with-pytorch)

Instance Segmentation (/mask-r-cnn-instance-segmentation-with-pytorch/)

In this post, we discuss image classification in PyTorch. We will use a subset of the [CalTech256](http://vision.caltech.edu/Image_Datasets/Caltech256/(opens%20in%20a%20new%20tab))) ([\(http://vision.caltech.edu/Image_Datasets/Caltech256/\(opens in a new tab\)\)](http://vision.caltech.edu/Image_Datasets/Caltech256/(opens in a new tab))) dataset to classify images of 10 animals. We will go over the steps of dataset preparation, data augmentation and then the steps to build the classifier. We use **transfer learning** to use the low level image features like edges, textures etc. These are learnt by a pretrained model, ResNet50, and then train our classifier to learn the higher level details in our dataset images like eyes, legs etc. ResNet50 has already been trained on ImageNet with millions of images. In another article on blog we discuss [Transfer Learning for Medical Images](https://learnopencv.com/transfer-learning-for-medical-images/) (<https://learnopencv.com/transfer-learning-for-medical-images/>).

Do not worry about functions and code. The post has code snippets to make it easy to study and understand. Also, the complete code has been made available over a python notebook (subscribe and download for free). Before we dive into the article, here's a video on [Pytorch image classification](https://www.youtube.com/watch?v=MOAkII27gh0) (<https://www.youtube.com/watch?v=MOAkII27gh0>) to motivate you further. Just like this video shows, you could build your own "Zoo classifier"!



While we have tried to make the post self-sufficient, we still encourage the readers to get familiarized to the [Basics of Pytorch](https://pytorch.org/docs/stable/index.html) (<https://pytorch.org/docs/stable/index.html>) before proceeding further.

Dataset Preparation

The [CalTech256](http://www.vision.caltech.edu/Image_Datasets/Caltech256/) (http://www.vision.caltech.edu/Image_Datasets/Caltech256/) dataset has 30,607 images categorized into 256 different labeled classes along with another 'clutter' class.

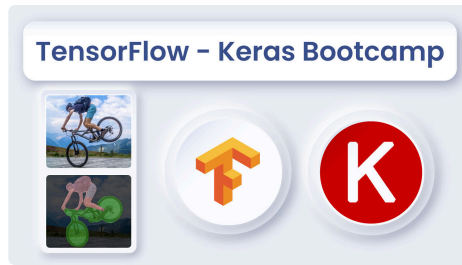
Training the whole dataset will take hours. So we will work on a subset of the dataset containing 10 animals – **bear, chimp, giraffe, gorilla, llama, ostrich, porcupine, skunk, triceratops, and zebra**. That way we can experiment faster. The code can then be used to train the whole dataset too.

The number of images in these folders varies from 81(for a skunk) to 212(for a gorilla). We use the first 60 images in each of these categories for training. The next 10 images are for validation, and the rest are for testing in our experiments below.

So finally, we have 600 training images, 100 validation images, 409 test images, and **10 classes** of animals.

If you want to replicate the experiments, please follow the steps below

- Download the [CalTech256](http://www.vision.caltech.edu/Image_Datasets/Caltech256/) (http://www.vision.caltech.edu/Image_Datasets/Caltech256/) dataset
- Create three directories with names **train, valid** and **test**.
- Create 10 sub-directories each inside the train and the test directories. The sub-directories should be named **bear, chimp, giraffe, gorilla, llama, ostrich, porcupine, skunk, triceratops** and **zebra**.
- Move the first 60 images for bear in the Caltech256 dataset to the directory train/bear. Repeat this step for every animal.
- Move the next 10 images for bear in the Caltech256 dataset to the directory valid/bear. Repeat this step for every animal.
- Copy the remaining images for bear (i.e. the ones not included in a train or valid folders) to the directory test/bear. Repeat this step for every animal.



Join Free TensorFlow Bootcamp

👤 15K+ Learners

🕒 3 Hours of Learning

Enroll for Free

(https://opencv.org/university/free-tensorflow-keras-course/?utm_source=locv&utm_medium=midblog&utm_campaign=learning-for-pytorch-image-classification)



Join Free PyTorch Bootcamp

👤 10K+ Learners

🕒 8 Hours of Learning

Enroll for Free

(https://opencv.org/university/free-pytorch-course/?utm_source=locv&utm_medium=midblog&utm_campaign=learning-for-pytorch-image-classification)

[View all AI Free Courses >](https://opencv.org/university/free-courses/?utm_source=locv&utm_medium=blog)
(https://opencv.org/university/free-courses/?utm_source=locv&utm_medium=blog)

Data Augmentation

The images in the available training set can be modified several ways to incorporate more variations in the training process. This way, the trained model gets more generalized and performs well on different test data. Also, the input data can come in a variety of sizes. They need to be normalized to a fixed size and format before batches of data are used together for training.

First, each of the input images is passed through many transformations. We try to insert some variations by introducing some randomness into the transformations. In each epoch, a single set of transformations are applied to each image. When we train for multiple epochs, the models see more variations of the input images with a new randomized variation of the transformation in each epoch. This results in data augmentation, and the model then tries to generalize more.

Below we see an example of the transformed versions of a Triceratops image.



Transformed versions of a Triceratops image

Let us go over the transformations we used for our data augmentation.

The transform *RandomResizedCrop* crops the input image by a random size (within a scale range of 0.8 to 1.0 of the original size and a random aspect ratio in the default range of 0.75 to 1.33). The cropped image is then resized to 256×256.

RandomRotation rotates the image by a random angle in the range of -15 to 15 degrees.

RandomHorizontalFlip randomly flips the image horizontally with a default probability of 50%.

CenterCrop crops a 224×224 image from the center.

ToTensor converts the PIL Image with values in the range of 0–255 to a floating point Tensor and normalizes them to a range of 0–1, by dividing it by 255.

Normalize takes in a 3 channel Tensor and normalizes each channel by the input mean and standard deviation for that channel. Mean and standard deviation vectors are input as 3 element vectors. Each channel in the tensor is normalized as $T = (T - \text{mean}) / (\text{standard deviation})$

All the above transformations are chained together using *Compose*.

Download Code To easily follow along this tutorial, please download code by clicking on the button below. It's FREE!

[Download Code](#)

```

1  # Applying Transforms to the Data
2  image_transforms = {
3      'train': transforms.Compose([
4          transforms.RandomResizedCrop(size=256, scale=(0.8, 1.0)),
5          transforms.RandomRotation(degrees=15),
6          transforms.RandomHorizontalFlip(),
7          transforms.CenterCrop(size=224),
8          transforms.ToTensor(),
9          transforms.Normalize([0.485, 0.456, 0.406],
10                               [0.229, 0.224, 0.225])
11      ]),
12      'valid': transforms.Compose([
13          transforms.Resize(size=256),
14          transforms.CenterCrop(size=224),
15          transforms.ToTensor(),
16          transforms.Normalize([0.485, 0.456, 0.406],
17                               [0.229, 0.224, 0.225])
18      ]),
19      'test': transforms.Compose([
20          transforms.Resize(size=256),
21          transforms.CenterCrop(size=224),
22          transforms.ToTensor(),
23          transforms.Normalize([0.485, 0.456, 0.406],
24                               [0.229, 0.224, 0.225])
25      ])
26  }

```

Note that for the validation and test data, we do not do the *RandomResizedCrop*, *RandomRotation* and *RandomHorizontalFlip* transformations. Instead, we resize the validation images to 256×256 and crop out the center 224×224 to be able to use them with the pretrained model. Finally, the image is transformed into a tensor and normalized by the mean and standard deviation of all the images in ImageNet.

Data Loading

Next, let us see how to use the above-defined transformations and load the data to be used for training.

```

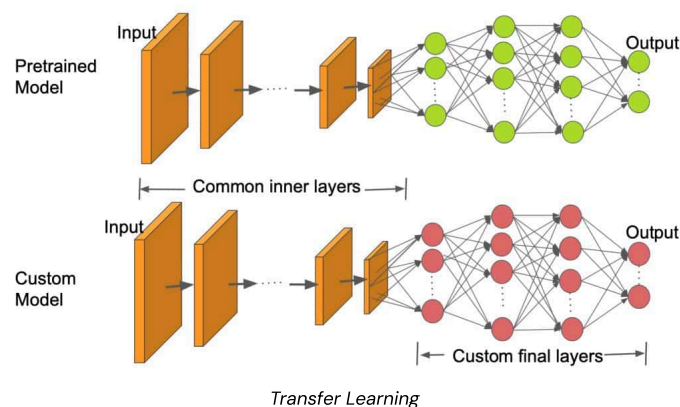
1  # Load the Data
2  # Set train and valid directory paths
3  train_directory = 'train'
4  valid_directory = 'test'
5  # Batch size
6  bs = 32
7  # Number of classes
8  num_classes = 10
9  # Load Data from folders
10 data = {
11     'train': datasets.ImageFolder(root=train_directory, transform=image_transforms
12     'valid': datasets.ImageFolder(root=valid_directory, transform=image_transforms
13     'test': datasets.ImageFolder(root=test_directory, transform=image_transforms['
14 }
15 # Size of Data, to be used for calculating Average Loss and Accuracy
16 train_data_size = len(data['train'])
17 valid_data_size = len(data['valid'])
18 test_data_size = len(data['test'])
19 # Create iterators for the Data loaded using DataLoader module
20 train_data = DataLoader(data['train'], batch_size=bs, shuffle=True)
21 valid_data = DataLoader(data['valid'], batch_size=bs, shuffle=True)
22 test_data = DataLoader(data['test'], batch_size=bs, shuffle=True)
23 # Print the train, validation and test set data sizes
24 train_data_size, valid_data_size, test_data_size

```

We first set the train and validation data directories and the batch size. Then we load them using **DataLoader**. Note that the image transformations we discussed earlier are applied to the data while loading them using the **DataLoader**. The order of the data is also shuffled. The **torchvision.transforms** package and the **DataLoader** are important PyTorch features that make the data augmentation and loading processes very easy.

Transfer Learning

It is very hard and time consuming to collect images belonging to a domain of interest and train a classifier from scratch. So, we use a pre-trained model as our base and change the last few layers to classify images according to our desirable classes. This helps us get good results even with a small dataset since the basic image features have already been learned in the pre-trained model from a much larger dataset like ImageNet.



As we can see in the above image, the inner layers are kept the same as the pretrained model and only the final layers are changed to fit our number of classes. In this work, we use the pre-trained ResNet50 model.

```

1 | # Load pretrained ResNet50 Model
2 | resnet50 = models.resnet50(pretrained=True)

```

[Canziani et al. \(https://arxiv.org/pdf/1605.07678.pdf\)](https://arxiv.org/pdf/1605.07678.pdf) list many pretrained models that are used for various practical applications, analyzing the accuracy obtained and the inference time needed for each model. ResNet50 is one model with a good tradeoff between accuracy and inference time. When a model is loaded in PyTorch, all its parameters have their 'requires_grad' field set to true by default. This means each and every change to the parameter values will be stored to be used in the backpropagation graph used for training. This increases memory requirements. Since most of the parameters in our pre-trained model are already trained, we reset the *requires_grad* field to false.

```

1 | # Freeze model parameters
2 | for param in resnet50.parameters():
3 |     param.requires_grad = False

```

Next, we replace the final layer of the ResNet50 model by a small set of Sequential layers. The inputs to the last fully connected layer of ResNet50 is fed to a Linear layer. It has 256 outputs, which are then fed into ReLU and Dropout layers. It is then followed by a 256×10 Linear Layer which has 10 outputs corresponding to the 10 classes in our CalTech subset.

```

1 | # Change the final layer of ResNet50 Model for Transfer Learning
2 | fc_inputs = resnet50.fc.in_features
3 | resnet50.fc = nn.Sequential(
4 |     nn.Linear(fc_inputs, 256),
5 |     nn.ReLU(),
6 |     nn.Dropout(0.4),
7 |     nn.Linear(256, 10),
8 |     nn.LogSoftmax(dim=1) # For using NLLLoss()
9 | )

```

Since we will be training on a GPU, we get the model ready for GPU.

```

1 | # Convert model to be used on GPU
2 | resnet50 = resnet50.to('cuda:0')

```

Next, we define the loss function and the optimizer to be used for training. PyTorch provides a variety of [loss functions \(https://pytorch.org/docs/stable/nn.html#loss-functions\)](https://pytorch.org/docs/stable/nn.html#loss-functions). We use the Negative Loss Likelihood function as it is useful for classifying multiple classes. PyTorch also supports multiple optimizers. We use the [Adam optimizer \(https://pytorch.org/docs/stable/optim.html#algorithms\)](https://pytorch.org/docs/stable/optim.html#algorithms). Adam is one of the most popular optimizers because it can adapt the learning rate for each parameter individually.

```

1 | # Define Optimizer and Loss Function
2 | loss_func = nn.NLLLoss()
3 | optimizer = optim.Adam(resnet50.parameters())

```

Training

The complete training code is in the python notebook, but we will discuss the main concept here. Training is carried out for a fixed set of epochs, processing each image once in a single epoch. The training data loader loads data in batches. In our case, we have given a batch size of 32. This means each batch can have a maximum of 32 images.

For each batch, input images are passed through the model, a.k.a **forward pass**, to get the outputs. Then the provided **loss_criterion** or cost function is used to compute the loss using the ground truth and the computed outputs. The gradients of the loss with respect to the trainable parameters are computed using the **backward** function. Note that with transfer learning, we need to compute gradients only for a small set of parameters that belong to the few newly added layers toward the end of the model. A summary function call to the model can reveal the actual number of parameters and the number of trainable parameters. The advantage we have in this approach is we now need to train only around a tenth of the total number of model parameters.

```
=====
Total params: 24,035,146
Trainable params: 527,114
Non-trainable params: 23,508,032
-----
```

Gradient computation is done using the **autograd** and **backpropagation**, differentiating in the graph using the chain rule. PyTorch accumulates all the gradients in the backward pass. So it is essential to zero them out at the beginning of the training loop. This is achieved using the optimizer's **zero_grad** function. Finally, after the gradients are computed in the backward pass, the parameters are updated using the optimizer's **step** function.

Total loss and accuracy is computed for the whole batch, which is then averaged over all the batches to get the loss and accuracy values for the whole epoch.

```
1  for epoch in range(epochs):
2      epoch_start = time.time()
3      print("Epoch: {}/{}".format(epoch+1, epochs))
4      # Set to training mode
5      model.train()
6      # Loss and Accuracy within the epoch
7      train_loss = 0.0
8      train_acc = 0.0
9      valid_loss = 0.0
10     valid_acc = 0.0
11     for i, (inputs, labels) in enumerate(train_data_loader):
12         inputs = inputs.to(device)
13         labels = labels.to(device)
14         # Clean existing gradients
15         optimizer.zero_grad()
16         # Forward pass - compute outputs on input data using the model
17         outputs = model(inputs)
18         # Compute loss
19         loss = loss_criterion(outputs, labels)
20         # Backpropagate the gradients
21         loss.backward()
22         # Update the parameters
23         optimizer.step()
24         # Compute the total loss for the batch and add it to train_loss
25         train_loss += loss.item() * inputs.size(0)
26         # Compute the accuracy
27         ret, predictions = torch.max(outputs.data, 1)
28         correct_counts = predictions.eq(labels.data.view_as(predictions))
29         # Convert correct_counts to float and then compute the mean
30         acc = torch.mean(correct_counts.type(torch.FloatTensor))
31         # Compute total accuracy in the whole batch and add to train_acc
32         train_acc += acc.item() * inputs.size(0)
33         print("Batch number: {03d}, Training: Loss: {:.4f}, Accuracy: {:.4f}"
```

Validation

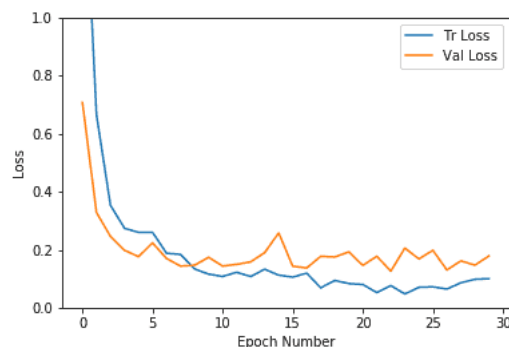
As training is carried out for more epochs, the model tends to overfit the data leading to its poor performance on new test data. Maintaining a separate validation set is important, so we can stop the training at the right point and prevent overfitting. Validation is carried out in each epoch immediately after the training loop. Since we do not need any gradient computation in the validation process, it is done within a `torch.no_grad()` block.

For each validation batch, the inputs and labels are transferred to the GPU (if cuda is available, else they are transferred to the CPU). The inputs go through the forward pass, followed by the loss and accuracy computations for the batch and at the end of the loop, for the whole epoch.

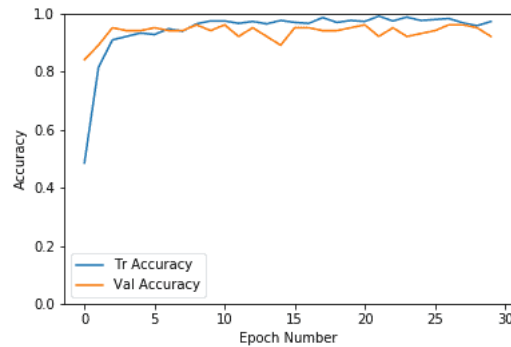
```

1  # Validation - No gradient tracking needed
2      with torch.no_grad():
3          # Set to evaluation mode
4          model.eval()
5          # Validation loop
6          for j, (inputs, labels) in enumerate(valid_data_loader):
7              inputs = inputs.to(device)
8              labels = labels.to(device)
9              # Forward pass - compute outputs on input data using the model
10             outputs = model(inputs)
11             # Compute loss
12             loss = loss_criterion(outputs, labels)
13             # Compute the total loss for the batch and add it to valid_loss
14             valid_loss += loss.item() * inputs.size(0)
15             # Calculate validation accuracy
16             ret, predictions = torch.max(outputs.data, 1)
17             correct_counts = predictions.eq(labels.data.view_as(predictions))
18             # Convert correct_counts to float and then compute the mean
19             acc = torch.mean(correct_counts.type(torch.FloatTensor))
20             # Compute total accuracy in the whole batch and add to valid_acc
21             valid_acc += acc.item() * inputs.size(0)
22             print("Validation Batch number: {:03d}, Validation: Loss: {:.4f}, /
23         # Find average training loss and training accuracy
24         avg_train_loss = train_loss/train_data_size
25         avg_train_acc = train_acc/float(train_data_size)
26         # Find average training loss and training accuracy
27         avg_valid_loss = valid_loss/valid_data_size
28         avg_valid_acc = valid_acc/float(valid_data_size)
29         history.append([avg_train_loss, avg_valid_loss, avg_train_acc, avg_valid_acc])
30         epoch_end = time.time()
31         print("Epoch : {:03d}, Training: Loss: {:.4f}, Accuracy: {:.4f}%, nttValid:

```



Loss curve for training and validation



Accuracy curve for training and validation

As we can see in the above plots, both the validation and training losses settle down pretty quickly for this dataset. The accuracy also increases up to the range of 0.9 very fast. As the number of epochs increases, the training loss decreases further, leading to overfitting, but the validation results do not improve much. So we chose the model from the epoch with higher accuracy and a lower loss. It is better if we stop early to prevent overfitting the training data. In our case, we chose epoch#8, which had a validation accuracy of 96%.

The **early stopping** process can also be automated. We can stop once the loss is below a threshold and if the validation accuracy does not improve for a given set of epochs.

Inference

Once we have the model, we can do inferences on individual test images or on the whole test dataset to obtain the test accuracy. The test set accuracy computation is similar to the validation code, except it is carried out on the test dataset. We have included the function [computeTestSetAccuracy](#) in the Python notebook for the same. Let us discuss below how to find the output class for a given test image.

An input image first undergoes all the transformations used for validation/test data. The resulting tensor is then converted to a 4-dimensional one and passed through the model, which outputs the log probabilities for different classes. An exponential of the model outputs provides us with the class probabilities. Then we choose the class with the highest probability as our output class.

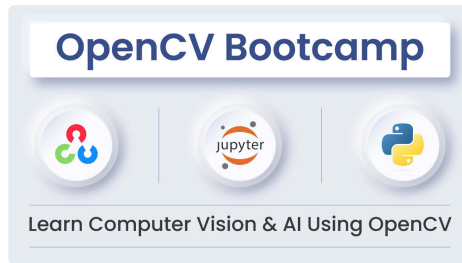
Choose the class with the highest probability as our output class.

```

1 def predict(model, test_image_name):
2     transform = image_transforms['test']
3     test_image = Image.open(test_image_name)
4     plt.imshow(test_image)
5     test_image_tensor = transform(test_image)
6     if torch.cuda.is_available():
7         test_image_tensor = test_image_tensor.view(1, 3, 224, 224).cuda()
8     else:
9         test_image_tensor = test_image_tensor.view(1, 3, 224, 224)
10    with torch.no_grad():
11        model.eval()
12        # Model outputs log probabilities
13        out = model(test_image_tensor)
14        ps = torch.exp(out)
15        topk, topclass = ps.topk(1, dim=1)
16        print("Output class : ", idx_to_class[topclass.cpu().numpy()[0][0]])

```

An accuracy of 92.4% was achieved on a test set with 409 images.

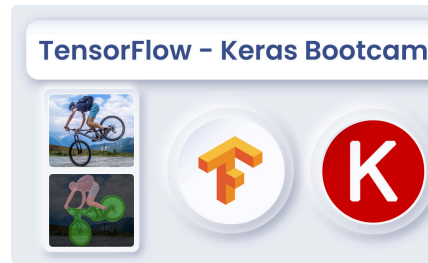
**Join Free OpenCV Bootcamp**

👤 100K+ Learners

🕒 3 Hours of Learning

Enroll for Free

(https://opencv.org/university/free-opencv-course/?utm_source=locv&utm_medium=midblog&utm_campaign=learning-for-pytorch-image-classification)

**Join Free TensorFlow Bootcamp**

👤 15K+ Learners

🕒 3 Hours of Learning

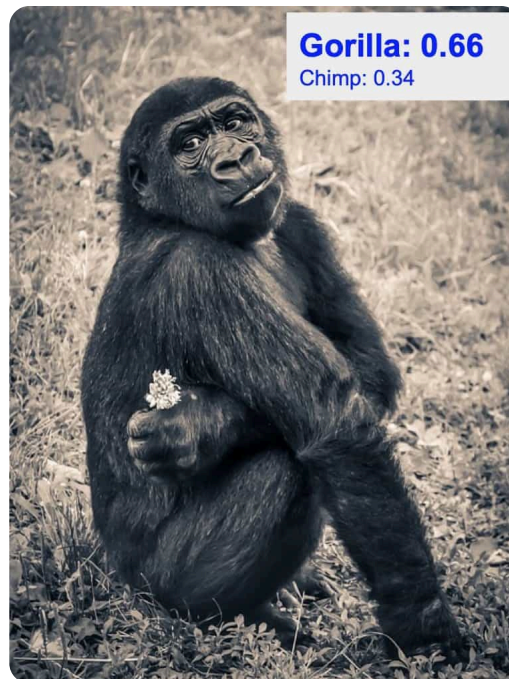
Enroll for Free

(https://opencv.org/university/free-tensorflow-course/?utm_source=locv&utm_medium=midblog&utm_campaign=learning-for-pytorch-image-classification)

[View all AI Free Courses >](#)

(https://opencv.org/university/free-courses/?utm_source=locv&utm_medium=blog)

Below are some of the classification results on new test data that were not used in training or validation. The top predicted classes for the images with their probability scores are overlaid on the top right. As we see below, the class predicted with the highest probability is often the correct one. Also, note that the class with the second highest probability is often the closest animal in appearance to the actual class amongst all the remaining 9 classes.









We just saw how to use a pretrained model trained for 1000 classes of ImageNet. It very effectively classified images belonging to the 10 different classes of our interest.

We showed the classification results on a small dataset. In a future post, we will apply the same transfer learning approach on harder datasets solving harder real-life problems. Stay tuned!

Acknowledgement

I would like to thank our intern Kushashwa Ravi Shrimali for writing the code for this post.

Also check out our series on [Getting Started With Pytorch](https://learnopencv.com/getting-started-with-pytorch/) (<https://learnopencv.com/getting-started-with-pytorch/>).

References

- Griffin, Gregory and Holub, Alex and Perona, Pietro (2007) *Caltech-256 Object Category Dataset*.
- Pixabay Images used in the blog post: [1] (<https://pixabay.com/photos/giraffe-africa-tanzania-wild-1330814/>), [2] (<https://pixabay.com/photos/dinosaur-dinosaur-world-theme-park-668612/>), [3] (<https://pixabay.com/photos/triceratops-dinosaur-statue-fossil-954293/>), [4] (<https://pixabay.com/photos/skunk-baby-mammal-brown-white-853084/>), [5] (<https://pixabay.com/photos/porcupine-echidna-nature-mammal-435458/>), [6] (<https://pixabay.com/photos/gorilla-monkey-puppy-ape-914585/>), [7] (<https://pixabay.com/photos/zebra-safari-wildlife-serengeti-171209/>), [8] (<https://pixabay.com/photos/bear-brown-bear-animal-mammal-422682/>), [9] (<https://pixabay.com/photos/llama-peru-nature-outside-ruins-90580/>), [10] (<https://pixabay.com/photos/chimpanzee-monkey-animal-world-3854044/>), [11] (<https://pixabay.com/photos/bouquet-cute-bird-poultry-feather-3320239/>), [11] (<https://pixabay.com/photos/porcupine-echidna-nature-mammal-435458/>)

Was This Article Helpful?



Subscribe & Download Code

If you liked this article and would like to download code (C++ and Python) and example images used in this post, please [click here](#). Alternately, sign up to receive a free [Computer Vision Resource](#) Guide. In our newsletter, we share OpenCV tutorials and examples written in C++/Python, and Computer Vision and Machine Learning algorithms and news.

 Download Example Code



(<https://learnopencv.com/kickstarter-ai/torch-model-inference-using-onnx-and-caffe2/>)

Load Comments

[AI \(https://learnopencv.com/tag/ai/\)](https://learnopencv.com/tag/ai/) [Computer Vision \(https://learnopencv.com/tag/computer-vision/\)](https://learnopencv.com/tag/computer-vision/) [deep learning \(https://learnopencv.com/tag/deep-learning/\)](https://learnopencv.com/tag/deep-learning/)

[Python \(https://learnopencv.com/tag/python/\)](https://learnopencv.com/tag/python/) [PyTorch \(https://learnopencv.com/tag/pytorch/\)](https://learnopencv.com/tag/pytorch/)

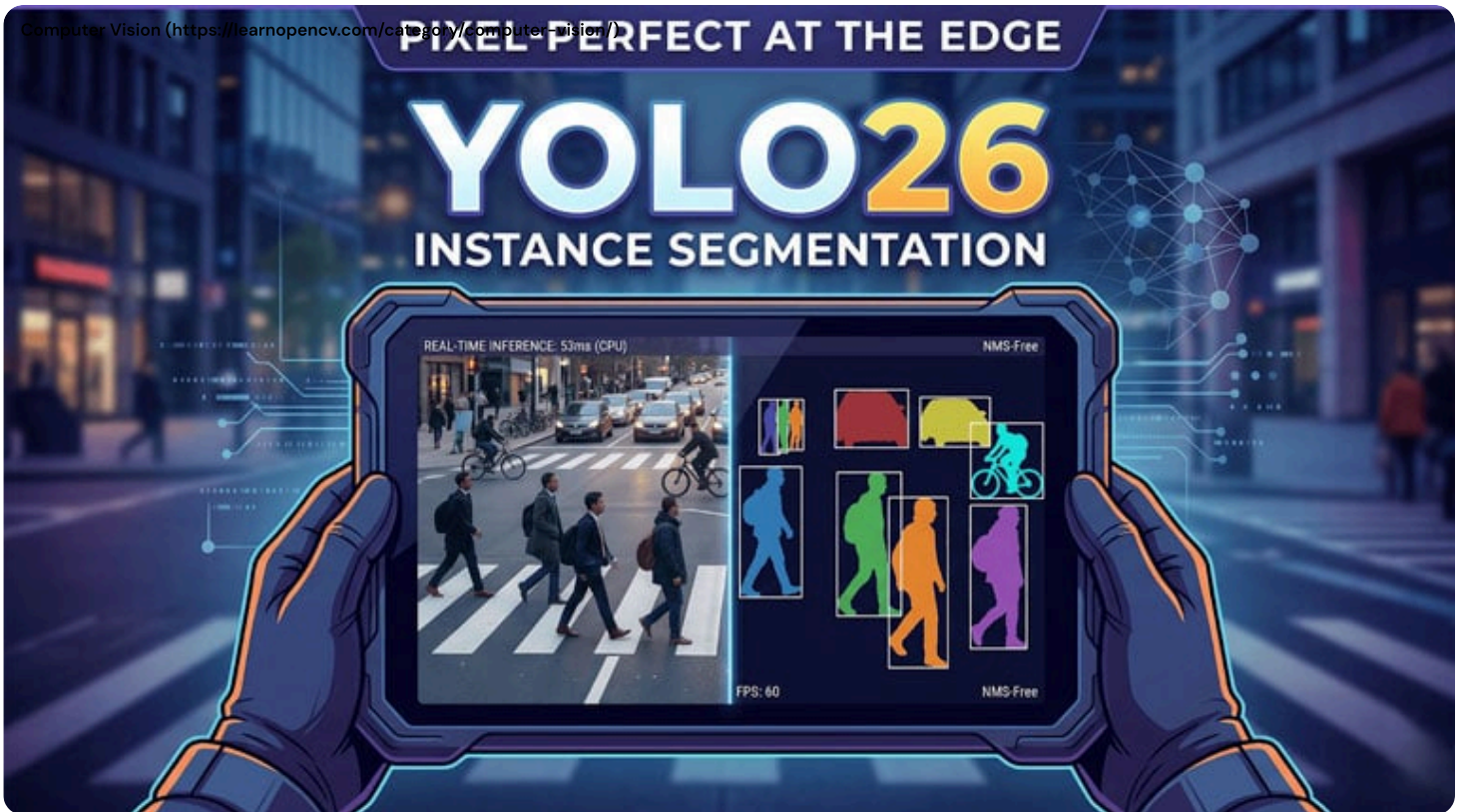
[pytorch transfer learning \(https://learnopencv.com/tag/pytorch-transfer-learning/\)](https://learnopencv.com/tag/pytorch-transfer-learning/)

[transfer learning machine learning \(https://learnopencv.com/tag/transfer-learning-machine-learning/\)](https://learnopencv.com/tag/transfer-learning-machine-learning/)

[transfer learning pytorch \(https://learnopencv.com/tag/transfer-learning-pytorch/\)](https://learnopencv.com/tag/transfer-learning-pytorch/) [transfer learning vs fine tuning \(https://learnopencv.com/tag/transfer-learning-vs-fine-tuning/\)](https://learnopencv.com/tag/transfer-learning-vs-fine-tuning/)

[what is transfer learning \(https://learnopencv.com/tag/what-is-transfer-learning/\)](https://learnopencv.com/tag/what-is-transfer-learning/)

Read Next



(<https://learnopencv.com/yolo26-instance-segmentation-pixel-perfect-ai-at-real-time-speed/>)

[Sudip Chakrabarty \(https://learnopencv.com/author/sudip/\)](https://learnopencv.com/author/sudip/) on March 26, 2026

YOLO26 Instance Segmentation: Pixel-Perfect AI at Real-Time Speed (<https://learnopencv.com/yolo26-instance-segmentation-pixel-perfect-ai-at-real-time-speed/>)

Build a complete pipeline for YOLO26 instance segmentation, from image and video inference to custom dataset training and edge deployment.



(<https://learnopencv.com/yolo26-nms-free-inference/>)

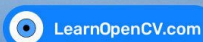
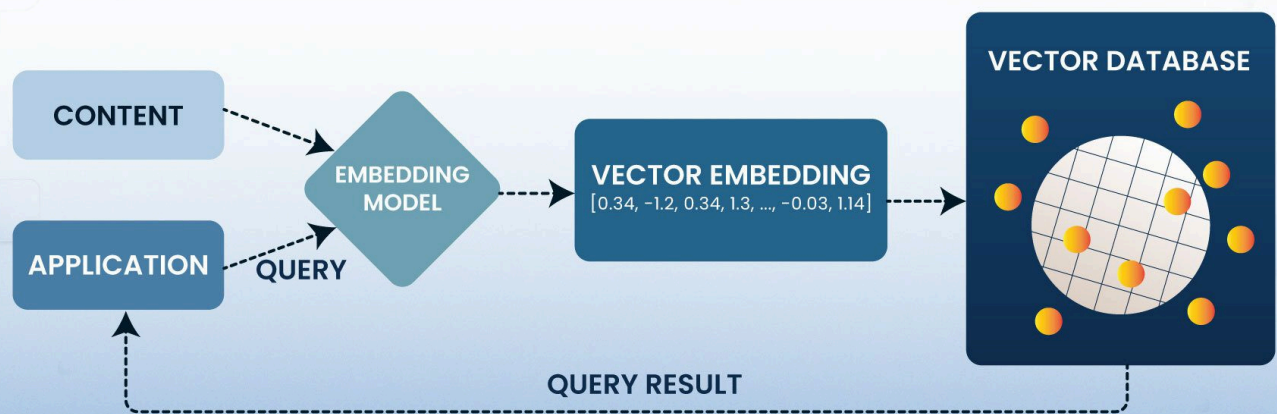
[Sudip Chakrabarty](https://learnopencv.com/author/sudip/) (<https://learnopencv.com/author/sudip/>) on February 10, 2026

Breaking the Bottleneck: Achieving Native NMS-Free Inference with YOLO26 (<https://learnopencv.com/yolo26-nms-free-inference/>)

YOLO26 introduces a paradigm shift with native NMS-free inference. Discover how its One-to-One label assignment eliminates post-processing overhead for stable,...

Generative AI (<https://learnopencv.com/category/generative-ai/>)

Ultimate Guide to Vector Databases and RAG pipeline



(<https://learnopencv.com/vector-db-and-rag-pipeline-for-document-rag/>)

<https://learnopencv.com/image-classification-using-transfer-learning-in-pytorch/>

Kukil (<https://learnopencv.com/author/kukil/>) on November 11, 2025

The Ultimate Guide to Vector DB and RAG Pipeline (<https://learnopencv.com/vector-db-and-rag-pipeline-for-document-rag/>)

Processing long documents with VLMs or LLMs poses a fundamental challenge: input size exceeds context limits. Even with GPUs, as...

Subscribe to our Newsletter

Subscribe to our email newsletter to get the latest posts delivered right to your email.

Send

About LearnOpenCV

Empowering innovation through education, LearnOpenCV provides in-depth tutorials, code, and guides in AI, Computer Vision, and Deep Learning. Led by Dr. Satya Mallick, we're dedicated to nurturing a community keen on technology breakthroughs.

[Read More \(https://learnopencv.com/about/\)](https://learnopencv.com/about/)

FREE Courses

- VLM Bootcamp
- PyTorch Bootcamp
- TensorFlow & Keras Bootcamp
- OpenCV Bootcamp
- Python for Beginners

Getting Started

- Installation
- PyTorch
- Getting Started with OpenCV
- Keras & Tensorflow

Categories

- Deep Learning
- Object Detection
- Image Classification
- YOLO
- Image Processing
- Image Segmentation

Big Vision

- AI Consulting
- Case Studies
- Get in Touch

Copyright © 2026 – BIG VISION LLC Privacy Policy (<https://learnopencv.com/privacy-policy/>) Terms and Conditions (<https://learnopencv.com/terms-and-conditions/>)

